CLIENT-SIDE ATTACKS

Sesión 03: XSS & CSRF

¿QUÉ ES UN ATAQUE CLIENT-SIDE?

- **Ejecución Local:** A diferencia de SQLi, el ataque ocurre en el navegador de la víctima, no directamente en el servidor.
- **Abuso de Confianza:** El navegador confía en el contenido que recibe de un dominio legítimo. El atacante inyecta contenido malicioso en ese flujo.
- **Objetivo Humano:** El fin último suele ser el robo de sesiones, suplantación de identidad o exfiltración de datos sensibles del usuario.
- **Tecnologías Clave:** JavaScript, HTML, CSS y el manejo de Cookies son los pilares de estas vulnerabilidades.

FUNDAMENTOS DE XSS

- **Definición:** Vulnerabilidad que permite a un atacante ejecutar scripts (JS) en el navegador de otros usuarios.
- **El concepto de Confianza:** El navegador no puede distinguir entre un script legítimo de la web y uno inyectado por un atacante.
- **Impacto:** Robo de `document.cookie`, secuestro de sesión (Hijacking), redirecciones forzadas y ataques de Keylogging.
- **Detección:** Se basa en encontrar parámetros de entrada que se reflejan en el HTML sin la debida sanitización o codificación.

TIPOS PRINCIPALES DE XSS

XSS REFLEJADO

El script malicioso proviene de la petición HTTP actual. El input del usuario se "refleja" inmediatamente en la respuesta. Requiere que la víctima haga clic en un enlace malicioso.

XSS ALMACENADO

El script se guarda permanentemente en el servidor (Base de Datos, Comentarios). Es más peligroso porque afecta a cualquier usuario que visite la página comprometida de forma automática.

LAB 01: REFLECTED XSS

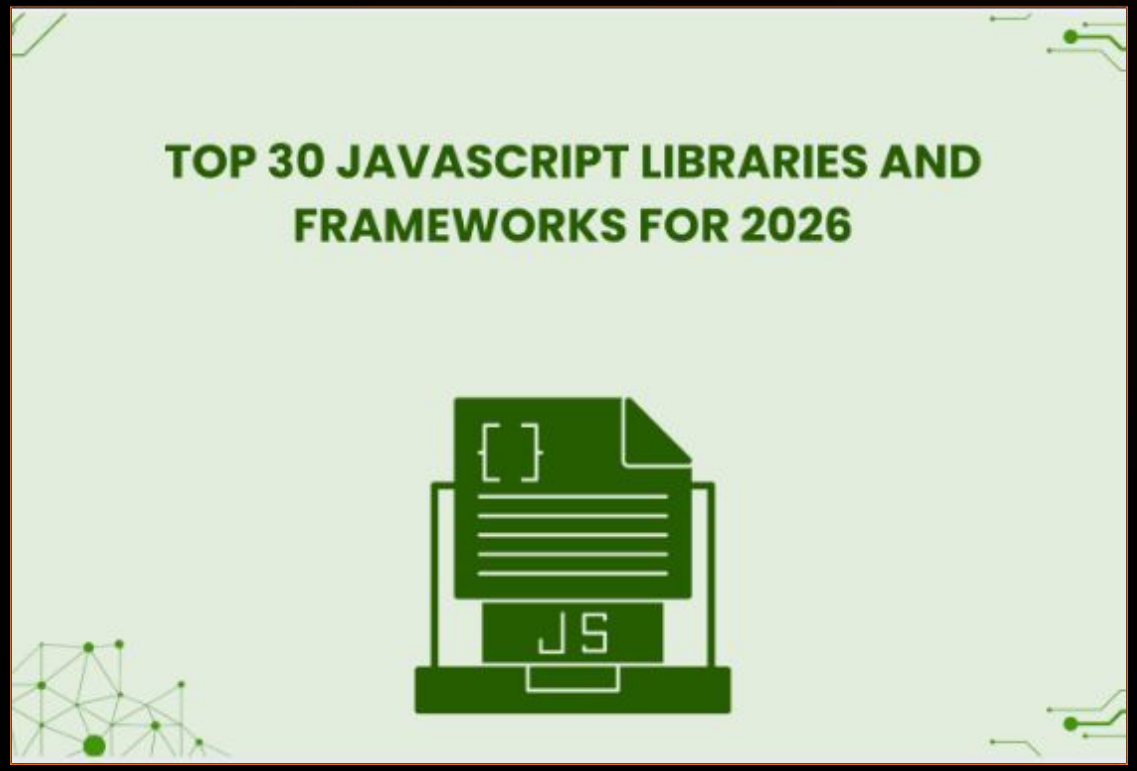
HTML Context con "Nothing Encoded".

ANÁLISIS TÉCNICO DEL LAB 01

En este escenario, la aplicación web cuenta con una función de **búsqueda**. El término buscado se refleja en la página de resultados dentro de una etiqueta HTML común.

Vulnerabilidad:

El servidor no aplica ninguna función de HTML Encoding. Caracteres como < y > pasan directamente al DOM, permitiendo inyectar nuevas etiquetas.



EXPLOTACIÓN PASO A PASO: LAB 01

1. **Reconocimiento:** Ingresamos una cadena de prueba como TEST123 en el buscador y observamos dónde se refleja.
2. **Verificación:** Inyectamos caracteres especiales < > " ' para ver si el servidor los escapa. En este lab, no lo hace.
3. **Inyección:** Introducimos el payload clásico en el buscador:

```
alert(1)
```

4. **Resultado:** El navegador interpreta la etiqueta, ejecuta el JS y lanza el popup. Hemos logrado **Reflected XSS**.

XSS EN TEMPLATE LITERALS

Hackeando la sintaxis moderna de ES6.

RIESGOS EN JS CONTEXTS

- **Contexto JavaScript:** El ataque ocurre cuando nuestro input cae dentro de una etiqueta `</div>` preexistente en la web.
- **Template Literals:** Cadenas de texto delimitadas por backticks (`) que permiten interpolación de expresiones.
- **Interpolación:** El uso de `${...}` permite ejecutar expresiones JS dentro de la cadena.
- **Bypass:** Si el desarrollador filtra comillas simples y dobles, pero no backticks, el Template Literal sigue siendo vulnerable.

LAB 02: UNICODE ESCAPED

Bypass de filtros avanzados en Template Literals.

ANÁLISIS DEL LAB 02

EL DESAFÍO

La aplicación refleja el input dentro de un script que usa Template Literals. Sin embargo, escapa caracteres como `<` `>` `'` `"` `\` `` ` `` usando secuencias Unicode (ej: \u0027).

EL BYPASS

A pesar del escapado Unicode, el motor de JavaScript procesa el bloque `${...}` antes de que los filtros de seguridad puedan neutralizar la lógica. La interpolación es la llave.

EXPLOTACIÓN PASO A PASO: LAB 02

- **Inspección:** Vemos que el input cae en: `Bienvenido, ${TU_INPUT}`.
- **Prueba de Filtro:** Intentamos cerrar el backtick pero vemos que se escapa como Unicode, rompiendo el ataque tradicional.
- **Ataque de Interpolación:** Inyectamos directamente la expresión de alerta dentro de la sintaxis de interpolación:

```
${alert(1)}
```

- **Éxito:** El navegador ejecuta lo que hay dentro de ${} sin necesidad de escapar de la cadena de texto original.

CROSS-SITE REQUEST FORGERY (CSRF)

Falsificación de Peticiones en Sitios Cruzados.

FUNDAMENTOS DE CSRF

- **Concepto:** Ataque que engaña al navegador de una víctima para que realice una acción no deseada en otra aplicación.
- **Mecanismo:** Los navegadores envían automáticamente las **Cookies de Sesión** en peticiones hacia el dominio asociado.
- **La Víctima:** Debe estar autenticada en el sitio vulnerable mientras visita el sitio malicioso del atacante.
- **Acción Sensible:** Transferencias, cambio de contraseñas o cambio de correos electrónicos.

REQUISITOS PARA CSRF

ACCIÓN RELEVANTE

Debe haber una petición que cambie el estado en el servidor (un cambio de configuración, no solo una lectura).

MANEJO DE SESIÓN

La aplicación debe depender únicamente de Cookies para identificar al usuario, sin tokens adicionales de validación.

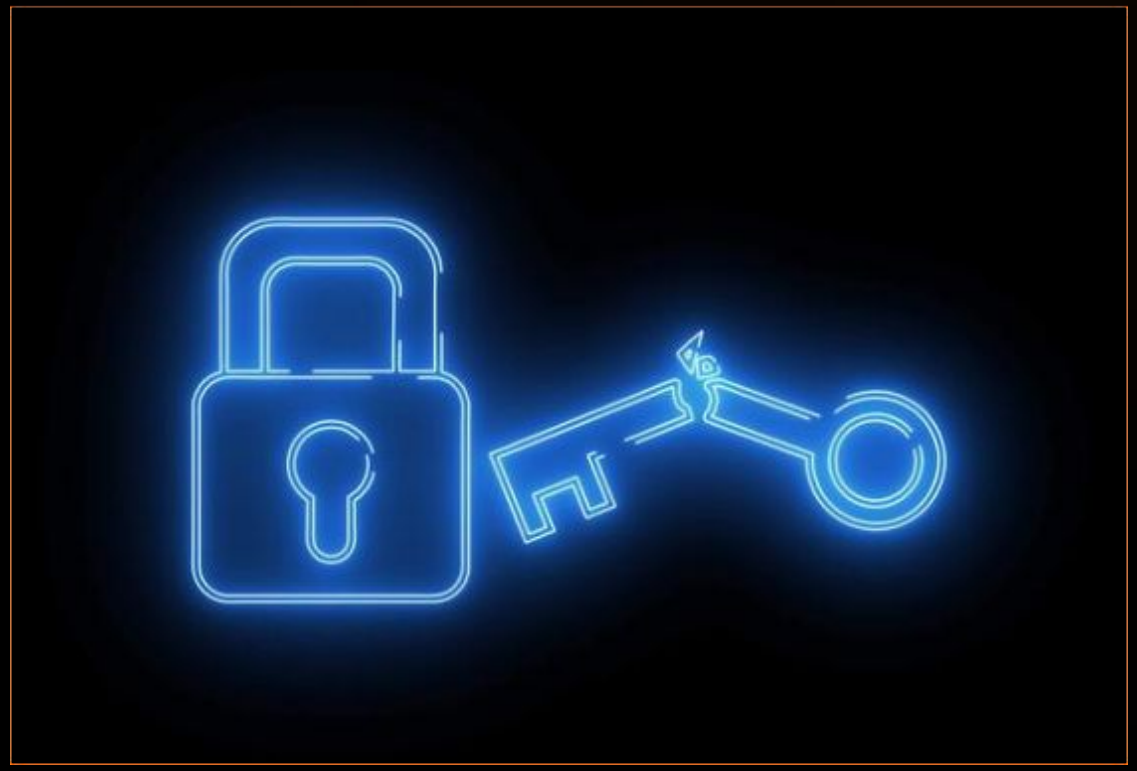
PRÁCTICA: CSRF NO DEFENSES

Caso de estudio: El cambio de Email.

ANÁLISIS DEL LAB: SIN DEFENSAS

En este laboratorio, el formulario de cambio de correo electrónico envía una petición **POST** sin ningún token de seguridad (Anti-CSRF).

Como atacantes, podemos construir un sitio web que contenga un formulario oculto que apunte a esa URL de cambio de email y que se envíe automáticamente mediante JS.



EXPLOTACIÓN PASO A PASO: CSRF BÁSICO

- **Interceptación:** Interceptamos la petición legítima de cambio de mail en Burp Suite para ver los parámetros.
- **Construcción:** Usamos el **Exploit Server** para alojar un HTML con un formulario auto-submit.

```
...
```

- **Envío:** Enviamos el enlace del exploit a la víctima (simulada por el bot del lab).
- **Impacto:** El navegador de la víctima hace el POST, envía su cookie, y el email cambia al nuestro.

DEFENSAS Y BYPASSES

Manipulación de tokens y métodos HTTP.

TOKENS ANTI-CSRF

EL CONCEPTO

Un valor aleatorio, único y secreto que el servidor incluye en el formulario. Al recibirlo de vuelta, valida que sea el correcto para esa sesión.

FALLA COMÚN

A veces el servidor solo valida el token si la petición es **POST**. Si la acción se permite vía **GET**, el token se ignora.

LAB: TOKEN DEPENDS ON METHOD

Bypass mediante Method Switching.

EXPLOTACIÓN: METHOD SWITCHING

- **Detección:** La petición POST requiere un token csrf válido.
- **Prueba de Cambio:** En Burp Suite, hacemos clic derecho y seleccionamos Change request method.
- **Eliminación:** Borramos el parámetro del token en la nueva petición GET.
- **Ejecución:** Si el servidor procesa el cambio de mail sin pedir el token en GET, la vulnerabilidad es explotable vía CSRF simple.

RESUMEN COMPARATIVO

Característica	XSS (Cross-Site Scripting)	CSRF (Cross-Site Request Forgery)
Ubicación Ejecución	Navegador del cliente (Script malicioso)	Navegador del cliente (Petición falsificada)
Mecanismo	Inyección de código en la web vulnerable	Abuso de la gestión de Cookies del navegador
Acceso a Datos	Total (puede leer Cookies, DOM, LocalStorage)	Limitado (puede actuar, pero no leer respuesta)
Defensa Principal	Sanitización y HTML Encoding	Tokens Anti-CSRF y SameSite Cookies

MITIGACIÓN Y MEJORES PRÁCTICAS

- **Output Encoding:** Codificar todo el contenido generado por el usuario antes de imprimirlo en el HTML.
- **Content Security Policy (CSP):** Header que restringe de dónde se pueden cargar y ejecutar scripts.
- **SameSite=Lax/Strict:** Atributo de Cookie que impide que se envíe en peticiones Cross-Site.
- **Tokens Anti-CSRF Fuertes:** Únicos por sesión, bien validados en **todos** los métodos.

CONCLUSIONES Y PRÓXIMOS PASOS

Cerrando la brecha en el Client-Side.

RESUMEN DE LA SESIÓN

- El XSS se trata de **ejecución**; el CSRF se trata de **acción**.
- Los filtros de seguridad modernos (como Unicode Escaping) pueden ser saltados con lógica de JS (Template Literals).
- Nunca confíes en que un Token CSRF es seguro si no has probado cambiar el método HTTP.
- La mentalidad del auditor es probar los límites de los filtros del desarrollador.

¡GRACIAS POR ASISTIR!

NOS VEMOS EN LA SESIÓN 04

Soy QB1t - Hack the Browser